Expert Systems With Applications 289 (2025) 128350

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Dynamic task scheduling optimization by rolling horizon deep reinforcement learning for distributed satellite system

Zhe Li [a,*], Xin Zhu [a], Changdi Liu [a], JunYao Song [a], Yubai Liu [b], Chengjun Yin [b], Wei Sun [a]

[a] College of Electrical and Information Engineering, Hunan University, Changsha, 410082, China
[b] China Academy of Space Technology, Beijing, 100094, China

ARTICLE INFO

Keywords:
Distributed multi-satellite system
Deep reinforcement learning
Multi-agent Markov decision process
Dynamic task scheduling

ABSTRACT

This paper investigates the dynamic task scheduling problem based on deep reinforcement learning for distributed multi-satellite system. To capture the task dynamics and optimize scheduling decisions, a model of multi-agent Markov decision process (MAMDP) is constructed, whose states include task arrival time, resource requirements, priorities, and satellite resource utilization, while the action space defines the scheduling choices of satellites within different time windows. Through continuous interactions with the environment, state transition information are obtained, and policy and value functions are developed by neural networks. Particularly, a rolling-horizon-based multi-agent proximal policy optimization (RH-MAPPO) algorithm is designed to adjust scheduling strategies in real-time, adapting to changes in task priorities and resource constraints. The configuration of centralized training but independent execution is adopted, which enhances the efficiency in global and local information application. Experimental results indicate that the proposed method outperforms existing deep reinforcement learning and heuristic algorithms in task completion rate, system revenue, and scheduling efficiency, achieving a 14.5 %–35.2 % reduction in scheduling time under 400 task scenarios, thereby verifying its efficiency and superiority in distributed satellite task scheduling.

1. Introduction

With the rapid advancement of space technology, the reliance on satellite resources in various fields such as Earth observation, communication and navigation, and scientific exploration has been continuously increasing, leading to an explosive development in satellite systems. Distributed multi-satellite system is becoming an important role in addressing complex space missions by its ability to cover broader geographical areas, provide flexible task scheduling, and enable efficient data acquisition. However, with the increasingly diverse and complex tasks, effective task scheduling within limited time and resource constraints has become a pressing issue to be resolved (Ferrari, Cordeau, Delorme, Iori, & Orosei, 2024; Zheng, Guo, & Gill, 2019). The challenge of the multi-satellite task scheduling (MSTS) problem stems not only from the growing number of satellites but also the diversity of tasks and the dynamic nature of the system operation environment.

Research in satellite task scheduling encompass both static and dynamic scenarios in domain. Recent advancements, particularly the application of deep reinforcement learning (DRL), reveal greater potential for scheduling algorithms to handle complex task scenarios (Liang et al., 2023; Song et al., 2024; Wang, Wu, Xing, & Pedrycz, 2020). Traditional satellite scheduling methods often rely on static or quasi-static task environments, assuming that all task information is fully known prior to scheduling and thus derives globally optimal scheduling decisions. Early static task scheduling primarily relied on combinatorial optimization algorithms, especially under conditions where task requirements and satellite resources were known as priory. Wolfe and Sorensen (2000) were the first to propose a window-constrained packing problem model, marking the first use of an optimization model to describe the satellite task scheduling problem. Based on this model, subsequent researchers developed various exact algorithms, such as branch-and-bound (Chu, Chen, & Tan, 2017; Dell'Amico, Montemanni, & Novellani, 2021; Dey, Dubey, & Molinaro, 2023; Zarpellon, Jo, Lodi, & Bengio, 2021) and dynamic programming (Chen, Hu, & Guo, 2023; Heydari, 2020; Lemaître, Verfaillie, Jouhaud, Lachiver, & Bataille, 2002), to address small-scale satellite task scheduling problems. He, Chen, Lu, Chen, and Wu (2019b) solved the Earth observation satellite scheduling problem using a density-based heuristic algorithm and demonstrated its optimality under certain assumptions. However, while these exact algorithms can provide optimal solutions, their computational complexity increases significantly as the problem scale grows, making them less feasible for real-time applications (Wu et al., 2024). To overcome

* Corresponding author.
E-mail addresses: zheli@hnu.edu.cn (Z. Li), wei_sun@hnu.edu.cn (W. Sun).

https://doi.org/10.1016/j.eswa.2025.128350
Received 3 December 2024; Received in revised form 24 April 2025; Accepted 24 May 2025
Available online 29 May 2025

this limitation, Zhang, Xing, Peng, Yao, and Chen (2019) proposed an adaptive large neighborhood search (A-ALNS) framework with an adaptive task assignment mechanism to efficiently handle the complexity of large-scale multi-satellite scheduling problems. Additionally, Cordeau and Laporte (2005) designed a hybrid mutation genetic algorithm for distributed planning in multi-satellite systems, which greatly enhanced task execution efficiency. Such studies demonstrate that heuristic and meta-heuristic algorithms may offer better adaptability and application prospects in large-scale problems.

However, in practical applications, task environments often exhibit significant dynamic characteristics, such as task arrival time, execution time, resource requirements, and priority variation during execution (Li, Sun, Fan, & Yang, 2024c). This dynamism not only increases the complexity of task scheduling but also imposes higher demands on the real-time performance and adaptability, making the dynamic scheduling a focus in current research (He, Liu, Chen, Xing, & Liu, 2019a). Unlike static task scheduling, dynamic task scheduling requires real-time optimization under conditions of incomplete or evolving task information (Mi, Song, Yang, Han, & Yuen, 2024). Chu et al. (2017) proposed a flexible branch-and-bound-based algorithm to improve scheduling efficiency for Earth observation satellites in maritime target recognition. In dynamic task scheduling problems, traditional exact algorithms often fail to effectively handle issues in complex task environments. To address this challenge, Cui, Han, Su, Zhang, and Guo (2020) proposed a multi-objective hyper-heuristic framework that integrates operation scheduling and resource allocation to optimize the flight deck scheduling of carrier-based aircraft. Qiu, Xu, Ren, and Teo (2021) designed a decoupled scheduling framework for agile satellite planning tasks using geometric approximation, dynamic estimation, and neural network fitting methods. This framework not only applies to satellite task scheduling but also offers a general solution for other complex scheduling problems. However, the aforementioned methods are largely based on manually crafted rules and human expertise, lacking the autonomous decision-making capabilities in handling uncertain tasks. For complex, fast, and responsive scheduling problems, they generally come with high computational costs.

In response to this, with the rapid development of artificial intelligence, particularly deep learning and reinforcement learning, the application prospects of DRL in solving complex dynamic optimization problems have garnered widespread attention (Kumar, Gaur, Chakravarthy, & Nanthaamornphong, 2025; Yuan, Peng, Sun, & Liu, 2023). The core of DRL lies in the agents' continuous interaction with the environment, gradually learning and optimizing decision-making strategies, thus enables adaptive decision-making in dynamically changing environments. This capability provides DRL with significant advantages in addressing the uncertainties and dynamics in MSTS (Chun et al., 2023; Ferreira et al., 2019; Li, Wang, Zhang, & Pan, 2024b). Compared with traditional scheduling methods, DRL-based scheduling approaches can adjust scheduling strategies in real time to adapt to changes in the task environment, avoiding the limitations of traditional algorithms that rely on manually set rules, thereby significantly improving the real-time performance and robustness of task scheduling. Additionally, DRL's self-learning capability makes it particularly effective in handling dynamic task combinations and multi-objective optimization, maximizing task success rates and overall system efficiency under limited resources (Sutton, 2018). Chen et al. (2020) designed an active resource management algorithm based on long short-term memory (LSTM) and DRL to solve the problem of radio resource management. This method improves the flexibility of resource allocation by learning temporal changes in the environment. In the field of satellite scheduling, He et al. (2020) proposed a novel general model framework using Q-learning networks, which not only effectively addressed the issue of long-term reward estimation in satellite task scheduling but also demonstrated excellent scalability across different task scenarios. In multi-satellite research, Dalin, Haijiao, Zhen, Yanfeng, and Shi (2020) introduced a multi-agent deep reinforcement learning (MADRL) approach to solve the real-time multi-satellite collaborative observation scheduling problem. Their study showed that MADRL can reduce communication cost and better utilize satellite resources. Moreover, Zhang et al. (2023) employed the multi-agent proximal policy optimization (MAPPO) algorithm to address task planning issues, achieving remarkable success in multi-scale task planning scenarios.

However, despite of the theoretical potential of DRL in solving the multi-satellite dynamic scheduling problems, its application still faces numerous challenges. For instance, key difficulties include how to design state representations, action spaces as well as reward functions that are well-suited to the characteristics of distributed multi-satellite system, and how to effectively address the training complexity and convergence issues of DRL in high-dimensional task spaces. Additionally, the challenge of collaborative scheduling in multi-satellite system, as well as the application of multi-agent reinforcement learning in this field, remain important directions that warrant further exploration (Ferreira et al., 2019).

In summary, DRL-based multi-satellite dynamic scheduling methods provide a novel technological approach to addressing the MSTS, offering significant theoretical value and broad application prospects. This paper focuses on the dynamic task scheduling problem in multi-satellite systems, constructing a multi-agent Markov decision process (MAMDP) model and proposing the rolling horizon-based multi-agent proximal policy optimization (RH-MAPPO) algorithm to solve it. Extensive experiments demonstrate that the proposed algorithm exhibits outstanding performance in handling complex dynamic tasks.

The main contributions of this paper are summarized as follows:

1. The designed framework of MAMDP incorporates the dynamic characteristics of tasks and represents the policy and value functions by neural networks, which addresses the uncertainties and incomplete information in multi-satellite dynamic task scheduling problems. Besides, by collecting state data through the interaction of satellites with the environment, this approach cleverly avoids the complex computations of the probability transition matrix typically encountered in conventional Markov decision process.
2. A centralized training and decentralized execution framework is employed based on the MAMDP model. Satellites collaborate to achieve system-wide goals in training and make good advantage of the global system resource. But in execution, local information are independently handled in a dynamic environment, saving real-time communication costs.
3. An innovative integration of rolling horizon strategies and MAPPO is developed as the RH-MAPPO algorithm, whose strategy can dynamically adjust scheduling plans within each time window. This approach enables real-time updates and optimization of the policy network during training, thereby maximizing task completion rates and system performance with remarkable superiority.

The rest of this paper is organized as follows: Section 2 elaborates the process to make the multi-satellite dynamic task scheduling problem as the MAMDP model. Section 3 presents the RH-MAPPO algorithm, which adapts task scheduling strategies in response to dynamic changes in task priorities and cancellations, ensuring efficient resource allocation and improved task completion rates in complex and uncertain environments. In Section 4, detailed model training experiments are proposed to validate the convergence and robustness of the proposed algorithm, as well as comparisons with heuristic and state-of-art multi-agent reinforcement learning methods to highlight its performance. Finally, Section 5 summarizes the paper and briefly outlines future research directions.

2. MAMDP-based dynamic scheduling for multi-satellite system

In this section, a detailed description of the multi-satellite dynamic task scheduling problem is provided, along with the presentation of a dynamic scheduling model based on the MAMDP.

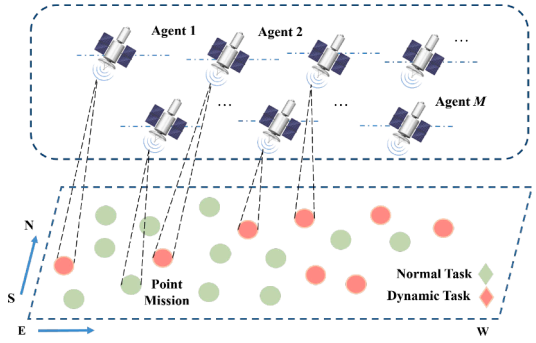

Fig. 1. Multi-satellite earth observation system architecture.

2.1. Problem description

The multi-satellite dynamic task scheduling problem is characterized by high uncertainty and dynamism (Dalin et al., 2020; Efrem & Panagopoulos, 2019; Fan et al., 2022; Wei, Xing, Wan, Song, & Chen, 2021). Specifically, dynamic tasks are tasks whose key attributes, such as arrival time, priority, and resource requirements, change over time. Dynamic task scheduling refers to the real-time process of assigning and scheduling these tasks to satellites, considering the time-varying nature of task characteristics and the availability of satellite resources (Li et al., 2024a). In this study, the framework of a multi-satellite Earth observation system is illustrated in Fig. 1. Each satellite is treated as an autonomous agent with independent computation and decision-making capabilities. These agents collaboratively manage complex observation and scheduling tasks.

In the specific scheduling process, observation requests from users are first received by ground-based satellite stations and preprocessed to generate a dataset of meta-tasks T, which includes data for satellites to process. Meta-task T encompasses several key attributes of task T_i, such as arrival time T_a^i, duration T_d^i, resource requirements T_{stor}^i, priority T_p^i, geographical location llh_i, and dynamics $u_i(t)$. Here, the dynamics attribute reflects potential state changes over time, such as task cancellations or adjustments in requirements. In this study, only the priority changes and task cancellations are taken into consideration. To systematically represent the dynamics $u_i(t)$ of task i, it is defined as a tuple containing multiple varying attributes such that,

$$u_i(t) = [T_{\Delta p}^{i,t}, T_c^{i,t}] \tag{1}$$

where $T_{\Delta p}^{i,t}$ represents the degree of dynamic change for task i at time step t, and $T_c^{i,t}$ indicates whether task i is canceled at time step t. This tuple can be simplified or extended based on specific experimental requirements. For instance, if at a certain time t, the priority of task T_i changes, its dynamic tuple can be represented as:

$$u_i(t) = [T_p^{i',t} = T_p^{i,t} + T_{\Delta p}^{i,t}, active] \tag{2}$$

where $T_p^{i',t}$ represents the priority of task i' after changes at time step t, while indicates that the task has not been canceled.

Thus, multi-satellite dynamic task scheduling can be summarized as follows: under specific constraints, each satellite autonomously allocates its resources to complete real-time observation tasks, aiming to maximize current mission objectives. Notably, Mi et al. (2024), Chu et al. (2017), Cui et al. (2020), Dai, Li, Fu, Zhao, and Chen (2020) extensively outlines the constraints for multi-satellite dynamic scheduling.

To ensure the feasibility and effectiveness of the scheduling plan, we consider a series of essential constraints. First, task uniqueness is a fundamental requirement, meaning that within the entire scheduling cycle T_{period}, each task T_i can be executed at most once. Second, a task's execution window must fall within its visible execution window. This implies that task T_i 's execution time $T_{start}^{i,j}$ on satellite S_j cannot begin before the task's arrival time T_a^i or the start of its visibility window D_{start}^j. Similarly, the task's end time $T_{end}^{i,j}$ cannot exceed either the task's deadline T_{end}^i or the end of its visibility window D_{end}^j. Additionally, for the same satellite S_j, the execution windows of different tasks must not overlap. This constraint requires tasks on a single satellite to be executed sequentially in time; if task T_i on satellite S_j ends at time $T_{end}^{i,j}$, then the start time $T_{start}^{k,j}$ of the next task $T_k$must follow. Finally, at any time step t, the resource utilization of each satellite must not exceed its current available resources $S_{stor}^j(t)$, ensuring that the satellite remains within its operational resource limits during task execution. Based on these constraints above, the expressions are illustrated as follows:

$$\sum_{j=1}^{M} \sum_{t=T_{start}^i}^{T_{end}^i} x_{i,j}(t) \le 1 \tag{3}$$

$$\begin{cases} T_{\text{start}}^{i,j} \ge \max\left(T_a^i, D_{\text{start}}^j\right) \\ T_{\text{end}}^{i,j} \le \min\left(T_{\text{end}}^i, D_{\text{end}}^j\right) \end{cases} \tag{4}$$

$$T_{end}^{i,j} - T_{start}^{k,j} \le 0, \ \forall i, j, k \quad j \ne k \tag{5}$$

$$\sum_{i=1}^{N} x_{i,j}(t) \times T_{stor}^i \le S_{stor}^j(t) \tag{6}$$

where $x_{i,j}(t)$ is a binary decision variable. If task T_i is selected to be executed at time t by satellite S_j, then $x_{i,j}(t) = 1$; otherwise, it is 0. Specifically:

$$x_{i,j}(t) = \begin{cases} 1, & \text{selected} \\ 0, & \text{unselected} \end{cases} \tag{7}$$

Considering the above constraints, the objective of this paper is to maximize the total reward (S_{total}) from successfully scheduled tasks, typically achieved by maximizing the cumulative rewards of the tasks. The objective function can be expressed as:

$$\max\left(V(S_{\text{total}}) = \sum_{j=1}^{M} V_j(S_j)\right) \tag{8}$$

where M is the number of satellites, and $V_j(S_j)$ represents the total expected reward obtained by satellite S_j.

2.2. Modeling process

In this study, to effectively address the dynamic characteristics of the problem, the model is constructed as MAMDP. Multi-agent Markov Decision Process (MAMDP) refers to a framework where multiple agents make decisions based on individual observations and actions, considering the overall system state. As shown in Fig. 2, the MAMDP framework optimizes the state evolution of the satellite system through a series of Markov decision processes executed by multiple agents, providing a strong theoretical foundation for applying the RH-MAPPO algorithm.

A traditional Markov decision process model includes elements such as state space, action space, state transition probability matrix, reward function, and value function (Eddy & Kochenderfer, 2020; Filar & Vrieze, 2012; He et al., 2020; Hu & Yue, 2007; Ren, Ning, & Wang, 2022). The state transition matrix typically defines the probability of transitioning from one state to another based on the agent's action. In this study, however, the transition matrix is not explicitly formulated. Instead, data are collected through interactions between the agents and the environment, enabling strategy optimization based on experience and avoiding to calculate or estimate transition probabilities. The MAMDP model for this study, based on the multi-satellite dynamic scheduling problem, is presented as follows:

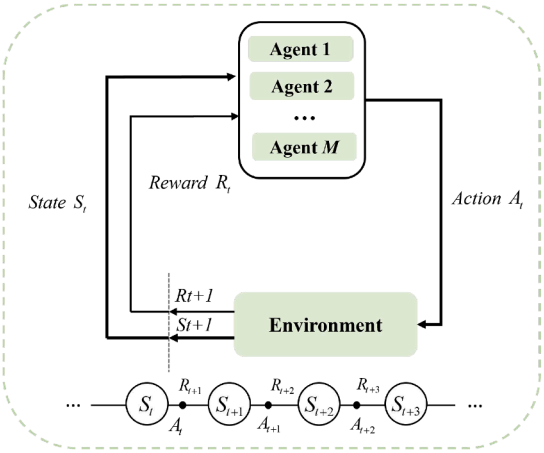

Fig. 2. Multi-agent environment interaction for multi-agent system dynamic scheduling.

2.2.1. State

The state space encompasses the joint states of all satellites. It can be expressed as follows:

$$S = \{S_0, S_1, \ldots, S_l, \ldots\} \tag{9}$$

where S_l represents the global state at time step t, composed of the local states O_k^t of all satellites. The detailed description is as follows,

$$S_l = \{O_1^t, O_2^t, \ldots, O_k^t, \ldots\} \tag{10}$$

In the above expression, O_k^t consists of a six-tuple that includes the remaining resource information and task information of the current satellite S_j, represented as follows:

$$O_k^t = [S_{time}^j, S_{stor}^j, T_{time}^{t,i}, T_{stor}^{t,i}, T_p^{t,i}, T_c^{t,i}] \tag{11}$$

where S_{time}^j represents the remaining free time t_{remain}^j for satellite S_j across all available time windows during the scheduling period. S_{stor}^j denotes the remaining storage capacity of satellite S_j. $T_{time}^{t,i}, T_{stor}^{t,i}$ refer to the task T_i arrival time and resource requirements provided by the user at the initial time step t respectively, which do not change over time. $T_p^{t,i}, T_c^{t,i}$ indicate the priority and cancellation status of task T_i respectively, which are dynamic parameters that may change over time t. The expression is as follows:

$$t_{remain}^j(t) = \bigcup_f (D_{end}^{j,f} - D_{start}^{j,f}) \tag{12}$$

where $D_{start}^{j,f}, D_{end}^{j,f}$ represent the start and end times of the fth visible time window for satellite S_j, respectively.

2.2.2. Action

The action space A is formed by the joint actions of all satellites, and its dimensionality expands with the increase in the number of satellites. It is defined as follows:

$$A = \{A_0, A_1, \ldots, A_l, \ldots\} \tag{13}$$

At each time step t, satellite S_k takes the corresponding scheduling action a_k based on its observed local state o_k^t and the learned policy $\pi_{\theta_k}(o_k^t)$. The policy function defines the probability distribution of selecting actions for the satellite at each state, and the joint policy of the entire multi-satellite system can be expressed as the product of the policies of all satellites:

$$\prod \pi_\theta(a|o) = \prod_{k=1}^{N} \left(\pi_{\theta_k}(a_k|o_k^t)|A_l = a_k, O_k^t = o_k^t\right) \tag{14}$$

where θ represents the parameters of the policy network. In this paper, the policy and value functions are explicitly represented using neural networks, employing the same network architecture and learning together to obtain an optimal policy network that maximizes expected returns. According to the Bellman optimality formula, the expression for the value function $v_\pi(s_k^t)$ is as follows:

$$v_{\pi_\theta}(s_{i,k}^t) = \mathbb{E}\left(R_l(s_{i,k}^t, a_{i,k}^t) + \gamma v_{\pi_\theta}(s_{i,k}^{t+1})|S_l = s_{i,k}^t\right) \tag{15}$$

In the expression above, $s_{i,k}^t$ represents the global state of satellite S_k completing the task in Section 3 at the current time step t, γ is the discount factor ($0 < \gamma \leq 1$), indicating that a higher value of γ places more emphasis on future rewards. $s_{i,k}^{t+1}$ denotes the global state at the next time step $t+1$, and R_l represents the immediate reward received by satellite S_l. The optimization of the satellite's policy network depends not only on the estimation of the value function but also on the immediate reward function at each time step t. The value function is based on the expectation of future cumulative rewards, while the immediate reward provides real-time feedback on the effectiveness of the satellite's actions after each decision. In this paper, the reward is designed based on the number of executable tasks, resource consumption of the satellites, and task priority, expressed as follows:

$$R_{i,j}^t = a\frac{T_p^{i,t}}{N^{i,t}} - b\frac{T_{stor}^{i,t}}{T_p^{i,t}} \tag{16}$$

where $N^{i,t}$ represents the number of satellites capable of observing task $T^{i,t}$ at time step t, a and b is the weight factor ($0 < a < 1, 0 < b < 1$), and $R_{i,j}^t$ denotes the immediate reward gained by satellite S_j for completing the task at time step t. The reward function enables the system to guide satellites to prioritize high-value tasks during the scheduling process while effectively managing resource consumption.

3. RH-MAPPO Method for dynamic task scheduling

To address the multi-satellite dynamic task scheduling problem discussed in the previous section, we propose an RH-MAPPO-based algorithm. As illustrated in Fig. 3, this framework leverages MAPPO as the core decision-making mechanism for each satellite within each rolling time window. Immediate rewards from interactions with the environment are used to evaluate decision effectiveness, while state information are stored in an experience replay buffer. Batches of samples from the buffer are utilized to estimate the value function and update the local policy networks of individual satellites. This adaptive framework efficiently responds to task changes and priority adjustments, ensuring dynamic flexibility and optimizing the overall system performance.

3.1. Multi-agent proximal policy optimization algorithm

The multi-agent proximal policy optimization algorithm extends the proximal policy optimization algorithm to address complex multi-agent scenarios. It has been widely applied in areas such as multi-UAV collaboration, robotic swarms, autonomous driving, and multi-satellite task scheduling (Hassan et al., 2023; Kang et al., 2023; Shi et al., 2023). In the context of multi-satellite dynamic task scheduling, each agent corresponds to a satellite, equipped with independent policy and value networks. Fig. 4 shows the Actor-Critic network structure in the MAPPO algorithm, highlighting the structure and input-output relationships of the actor and crtic networks, with color-coded information flows for clarity.

The policy network takes the satellite's local state (Eq. (11)) as input and outputs the corresponding action. For the value network, the input is a global state, which integrates the joint states of all satellites into a large tensor (Eqs. (9)-(10)), producing the value of the current policy (Eq. (15)). Both networks share consistent hidden layer parameters and employ relu activation functions to ensure efficient information propagation.

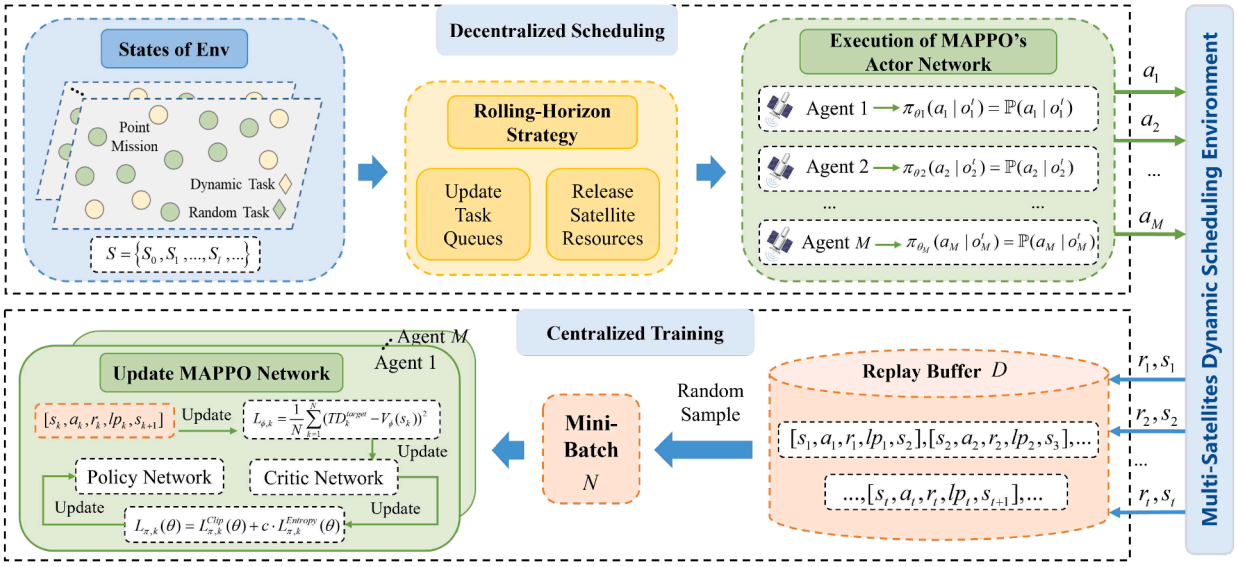

Fig. 3. The framework of the RH-MAPPO.

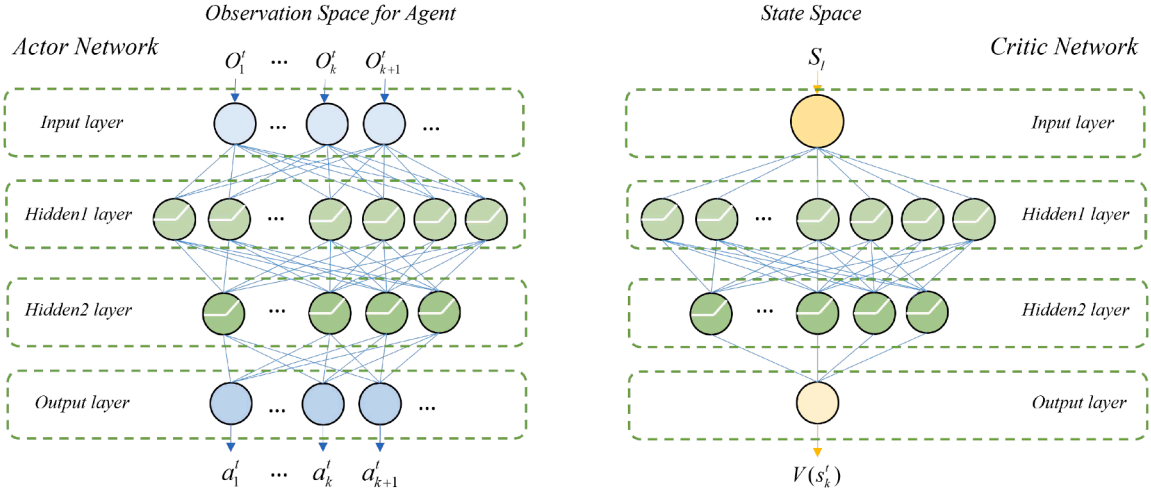

Fig. 4. Structure of actor (policy) networks and critic (value) networks in MAPPO.

As shown in Fig. 3, the framework adopts a centralized training and decentralized execution mechanism. During the training phase, satellites use global state information to evaluate policies, while in the execution phase, each satellite makes independent decisions based on its local policy network. This approach follows a fully cooperative model, where collaboration between satellites is achieved through information sharing and global optimization, effectively avoiding conflict resolution issues and simplifying the decision-making process. Algorithm 1 outlines the MAPPO workflow proposed in this study, with subsequent sections providing detailed descriptions of its key components.

3.1.1. Training process

In Algorithm 1, the training process simulates the task scheduling environment by providing various scenarios, enabling satellites to learn cooperation and decision-making in different contexts. For each training scenario, the environment state and each satellite's observations are initialized. Satellites generate actions based on their respective policy networks in response to the current observation. The policy network outputs the current action along with its corresponding log probability log_k^t. Specifically, log_k^t is calculated using the policy network based on the current state o_k^t and action a_k^t:

$$log_k^t = \log(\pi_\theta(a_k^t|o_k^t)) \tag{17}$$

The recording of log_k^t aids in calculating the ratio in the subsequent policy update, allowing for the assessment of the similarity between the current and previous policies. Then, in each training scenario, satellites accumulate experiences through interaction with the environment. After executing an action, the system returns the corresponding reward, the next state, and a flag indicating whether the satellite has completed its task. This information is stored in the experience pool for future training use.

3.1.2. Update actor-network and critic-network with experiment replay

In the process of updating the policy and value networks, experience replay is utilized to enhance sample efficiency. Data batches are randomly sampled from the experience pool, enabling effective network updates. The updates for the policy and value networks are closely integrated, ensuring collaborative optimization during training. In this study, the policy network update is formulated as a combination of two loss functions, specifically:

$$L_{\pi,k}(\theta) = L_{\pi,k}^{Clip}(\theta) + c \cdot L_{\pi,k}^{E}(\theta) \tag{18}$$

where c represents the weight coefficient for the entropy loss ($0 < c < 1$). $L_{\pi,k}^{Clip}(\theta)$ is the clipped loss function that limits the magnitude of policy updates to ensure the stability of policy optimization, and $L_{\pi,k}^{E}(\theta)$ is the

Algorithm 1 The Multi-Agent Proximal Policy Optimization Algorithm.

Initialize: Actor network and Critic network for each agent with weights θ_π and ϕ_v, respectively.

for each *training scenario* **do**
  Import a new scenario
  Initialize the replay memory D
  Initialize environment state S_0
  for each episode **do**
    for each agent $k = 1$ to M **do**
      Sample action $a_k^t \sim \pi_\theta(a_k^t|o_k^t)$, with log probability log_probs log_k^t based on (17)
      Execute action a_k^t, observe reward r_k^t, next state s_k^{t+1}, next observation o_k^{t+1}, and done d_k^t
      Store $(s_k^t, a_k^t, log_k^t, r_k^t, s_k^{t+1}, o_k^{t+1}, d_k^t)$ in D
      Update current state $s_k^t \leftarrow s_k^{t+1}$
    end for
    Sample a random batch from replay memory D
    for each agent $k = 1$ to M **do**
      Calculate ratio $R_k^t(\theta_\pi)$ and GAE A_k^t based on (20) and (21)
      Compute loss $L_\pi(\theta_\pi)$ and $L_\phi(\phi_v)$ using (18) and (23)
      Update Actor network and Critic network: $\theta \leftarrow \theta - \alpha\nabla_\theta L_\pi(\theta_\pi)$, $\phi \leftarrow \phi - \beta\nabla_\phi L_v(\phi_v)$
    end for
  end for
end for

entropy penalty loss function, which prevents the policy from prematurely converging to a deterministic action, thereby encouraging exploration.

Here is the clipping function used to compute the loss for the policy network:

$$L_{\pi,k}^{CLIP}(\theta) = -\mathbb{E}\min(R_k^t \cdot \hat{A}_k^t, \text{clip}(R_k^t, 1-\varepsilon, 1+\varepsilon)\hat{A}_k^t)] \tag{19}$$

where ε represents the clipping parameter $(0 < \varepsilon < 1)$, and R_k^t denotes the ratio of the new to old policy, which is the logarithmic probability of a particular action under both the new and old policies.

$$R_k^t = \frac{\pi_\theta(a_k^t, o_k^t)}{\pi_{\theta old}(a_k^t, o_k^t)} = exp(log_k^{new} - log_k^{old}) \tag{20}$$

Besides, $\hat{A}_k^t$ denotes the generalized advantage estimation (GAE), which combines the benefits of Monte Carlo methods and temporal difference methods. By smoothing future returns, GAE reduces variance. The formula is as follows:

$$\begin{aligned} \delta_t &= r_k^t + \gamma V\left(s_k^{t+1}\right) - V\left(s_k^t\right) \\ \hat{A}_k^t &= \sum_{l=0}^{\infty}(\gamma\lambda)^l \delta_k^{t+1} \end{aligned} \tag{21}$$

where r_k^t represents the instantaneous reward of satellite S_k at time step t, and λ is the parameter used to control the trade-off between bias and variance.

The entropy penalty loss function is given as:

$$L_{\pi,k}^{E}(\theta) = -\mathbb{E}\left[-\sum\nolimits_{a_k^t} \pi_\theta(a_k^t, o_k^t)\log\pi_\theta(a_k^t, o_k^t)\right] \tag{22}$$

The objective of the critic network is to minimize the difference between the estimated state value and the actual return. The Mean Squared Error loss function is adopted, such that

$$L_{\phi,k} = \frac{1}{N}\sum_{k=1}^{N}\left(TD_k^{target} - V_\phi(s_k)\right)^2 \tag{23}$$

where, N is the batch size, r_k^t is the current reward, and TD_k^{target} is the temporal difference target value:

$$TD_k^{target} = r_k^t + \gamma(1-d_k^t)V_\phi(s_k^{t+1}) \tag{24}$$

where, d_k^t indicates whether the satellite's state has ended; if it has ended $d_k^t = 1$, the value of the next state $V_\phi(s_k^{t+1})$ will no longer influence the current temporal difference target value TD_k^{target}.

3.2. Rolling horizon strategy algorithm

In this section, we propose a rolling horizon strategy method to address the dynamic complexity of satellite task scheduling. As a common approach in dynamic scheduling and decision-making problems, rolling horizon strategies have been extensively studied in various fields, including production scheduling, logistics management, and traffic management (Campuzano-Bolarín, Mula, Díaz-Madroñero, & Legaz-Aparicio, 2020; Gkiotsalitis, 2021; Wu, Dalle Ave, Harjunkoski, & Imsland, 2021). Given the unpredictability of task arrivals and the time-varying nature of task priorities and states, the rolling horizon strategy divides the scheduling process into smaller, more manageable time windows. Within each window, decisions are made based on the current state, allowing the system to maintain flexibility in response to task changes and adapt its scheduling strategy to the evolving dynamics of the environment.

The rolling horizon strategy framework in this paper is illustrated in Fig. 5. Within the defined total time range for dynamic task scheduling, this range is divided into several rolling horizons W_h^{RH}, which progress over time according to the rolling interval size W_I^{RH}. Taking Sat 2 as an example, as the window moves, the system updates the current task queue Q_T and the satellite's resource states S_{time}^j and S_{stor}^j based on the different dynamic changes of the tasks, and then executes Algorithm 1 to dynamically adjust the subsequent scheduling plan. The end W_{he}^{RH} of each rolling horizon window marks the beginning of a new round of scheduling optimization. The task queue is expressed as follows:

$$Q_T(t) = \{q_1(t), q_2(t), \ldots, q_i(t), \ldots\} \tag{25}$$

where $Q_T(t)$ represents the task queue at time step t, while $q_i(t)$ denotes the sorting position of task T_i within the queue. The order of tasks is determined by the magnitude of their priority changes $T_{\Delta p}^{i,t}$, arranged in descending order from left to right. A higher priority indicates a greater urgency for the task.

Within the rolling window, the task types include dynamic tasks T_D^i, regular tasks T_N^i, canceled tasks T_C^i, and executed tasks T_F^i. Changes in environmental factors and user demands often lead to alterations in task priorities $T_{\Delta p}^{i,t}$ and task cancellations $T_c^{i,t}$, necessitating real-time adjustments to the current scheduling strategy. The specific process is outlined in Algorithm 2, and the following sections will introduce the main components of the rolling horizon strategy presented in this paper.

Algorithm 2 The Rolling Horizon Multi-Agent Proximal Policy Optimization Algorithm.

Initialize: Rolling horizon W_h^{RH}, rolling interval W_I^{RH}.

while $W_{he}^{RH} < T_{max}^{period}$ **do**
  for each step t **do**
    Update task queues $Q_T(t)$ based on (26)
    Release resources $S_{time}^{j,t}, S_{stor}^{j,t}$ based on (27)
    Execute Algorithm 1 based on (28)
  end for
  Advance the rolling horizon: $W_{h+1}^{RH} = W_h^{RH} + W_I^{RH}$
end while

3.2.1. Update task queues

For tasks $T_{\Delta p}^{i,t}$ with changing priorities, the system will reassess their ranking position $Q_T(t)$ in the task queue based on the new task attributes and dynamically adjust resource allocation to ensure that high-priority tasks are executed first. As time progresses, two scenarios can arise such that the new tasks arrive and change the priorities of existing tasks. Thus, the update of the task sequence includes two components:

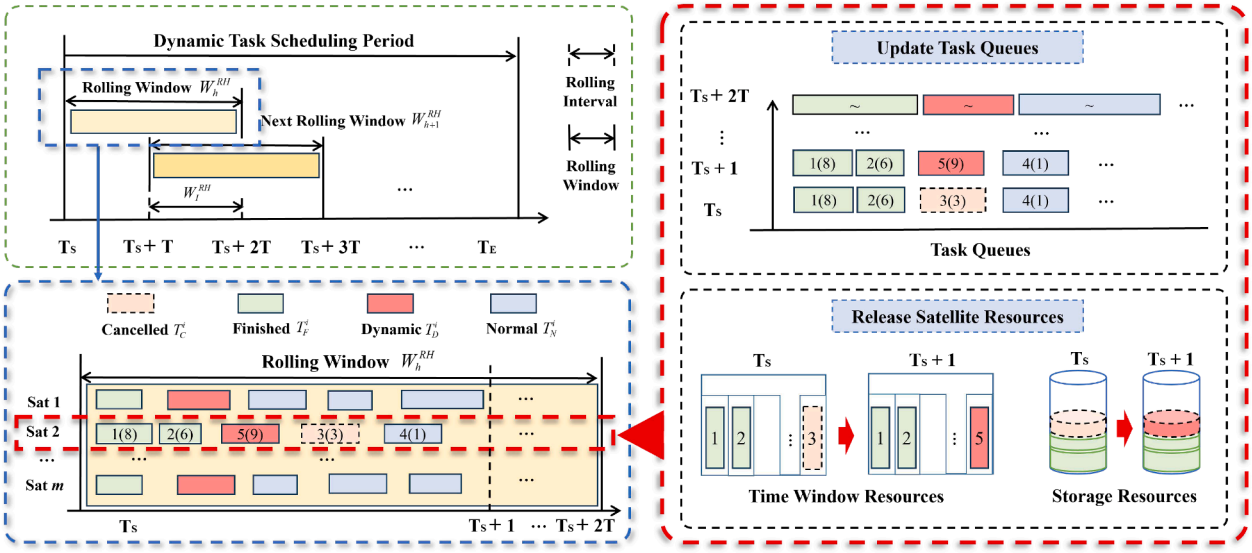

Fig. 5. The framework of rolling horizon strategy.

- Arrival of New Tasks: At time step $t+1$, new task $T_p^{i',t+1}$ may arrive and need to be incorporated into the task sequence.
- Priority Updates: The priorities $T_{p'}^{i,t+1}$ of existing tasks will change dynamically over time.

The expression for updating the task sequence is defined as follows:

$$Q_T(t+1) = Sort(Q_T(t) \cup \{T_p^{i',t+1}\}, by\ p_i(t+1)) \tag{26}$$

where, at time step $t+1$, the new sequence $Q_T(t+1)$ consists of existing tasks $Q_T(t)$ and newly arrived tasks, sorted according to the priority levels $p_i(t+1)$ of the tasks.

3.2.2. Release satellite resources

For canceled tasks $T_C^{i,t}$, the system will immediately release the satellite's visible time window resources $S_{time}^{j,t}$ and storage resources $S_{stor}^{j,t}$ that were allocated to the task. The expression is as follows:

$$S_{released}^{j,i}(t) = T_C^{i,t} \cdot (S_{time'}^{j,t} + S_{stor'}^{j,t}) \tag{27}$$

where $S_{released}^{j,i}(t)$ represents the resources $S_{time'}^{j,t}$ and $S_{stor'}^{j,t}$ released by satellite S_j for the canceled task $T_C^{i,t}$ at time step t. Using the resources released from canceled tasks, the satellites will reallocate these resources to other pending tasks within the rolling time window, ensuring maximum scheduling efficiency and preventing resource waste.

3.2.3. Execute algorithm 1

Once the resources and task sequence are determined, the next step is to perform online decision-making for tasks using Algorithm 1. Within the rolling time window, this information serves as input to Algorithm 1. The algorithm utilizes its built-in policy network, combining real-time environmental states with characteristics such as task priority and resource requirements, to dynamically allocate resources for the remaining tasks through the reward function (Eq. (16)). According to the policy of Algorithm 1, the task decision at the current time can be expressed as:

$$\pi_{\theta_j}(a_j|o_j^t) = (a_j|o_j^t) \tag{28}$$

where π_{θ_j} represents the probability of action a_j outputted by the policy network of the jth satellite in Algorithm 1 given the state o_j^t.

In Algorithm 2, this study integrates the rolling horizon strategy with the DRL method, enabling the system to dynamically respond to changes in task priorities and cancellations while continuously optimizing task assignments. This approach allows the satellite system to swiftly adapt to dynamic task changes, adjusting strategies to allocate resources efficiently and maximize overall system profit.

4. Simulation study

In this section, the proposed RH-MAPPO algorithm is applied to dynamic scenarios of varying scales. Numerical simulations are conducted to compare its performance with other multi-agent reinforcement learning algorithms and heuristic algorithm, thereby validating the effectiveness of the proposed method.

4.1. Design of the scenarios

4.1.1. Scenario description

The multi-satellite system includes optical imaging satellites and radar imaging satellites, with a maximum data storage capacity of 200 GB and orbital altitudes ranging from 800 to 1000 km. The data range for the satellite orbital parameters is presented in Table 1.

In this experiment, the scheduling cycle for the scenarios was set to one hour. A program was designed to simulate task scheduling scenarios, where tasks were represented as point targets randomly distributed between 3°N to 53°N latitude and 74°E to 133°E longitude. The visibility time windows for each satellite were calculated based on the geographical locations of the targets and satellite orbital parameters. Task imaging times were set between 3 and 6 s, accounting for precision requirements and satellite hardware capabilities. Each task required between 3 GB and 6 GB of storage, reflecting variations in complexity and data volume. To account for the importance of different tasks, a profit differentiation scheme was implemented, establishing a profit range from level 1 to 10.

4.1.2. Scenario parameters

Considering the variations in the number of satellites, scenarios, tasks, and rolling window intervals, we designed multiple experiments to validate the effectiveness of the proposed RH-MAPPO algorithm. The training parameters for the algorithm's network are presented in Table 2.

The other parameters of the scenarios are introduced as follows:

1. The discount factor γ in network training: 0.95.
2. Number of experience has stored D: 100000.

Table 1
Orbital elements of the satellite.

$a(km)$	$e(°)$	$i(°)$	$\Omega(°)$	$\omega(°)$	$m(°)$
6800~7200	0	98	290~310	40~165	250~320

Table 2
Network parameters of the scenarios.

Scenarios	satellites	HL1	HL2	batch N
300_50	4	128	100	64
300_100	4	128	100	64
300_150	4	128	100	64
500_100	7	128	100	128
500_200	7	128	100	128
500_300	7	128	100	128
500_400	7	128	100	128

3. Entropy loss weight factor c of Actor networks: 0.02.
4. Clipping factor ε of Actor network: 0.2.
5. Learning rate α of Actor network: 0.01.
6. Learning rate β of Critic network: 0.01.

4.1.3. Experiment conditions

The satellite and task data used in this study were sourced from the satellite tool kit (STK) software. The experiments were conducted on a Windows operating system with PyCharm as the development environment, utilizing Python and the PyTorch deep learning framework for model development and training. The experimental setup included a laptop equipped with an AMD Ryzen 7 6800H processor (3.20 GHz) and a dedicated GeForce GTX 2050 graphics card with 4 GB of memory.

4.2. Algorithmic effectiveness

This section presents experiments with varying task scales to validate the convergence of the RH-MAPPO algorithm. The rolling window size was set to 10 min, with task scales of 50, 100, and 150. The training aimed to observe convergence behavior across these task scales. Fig. 6 shows the total rewards, number of completed tasks, and average time per scenario for the RH-MAPPO algorithm at different task scales.

During the initial training phase (Scenarios 1-10), the RH-MAPPO algorithm adopted a greedy strategy, attempting to schedule all feasible tasks, which resulted in a high completion rate but also task oversubscription. However, task rewards were not effectively optimized in this phase, as the algorithm struggled to prioritize high-reward tasks, leading to lower overall rewards.

As training progressed (Scenarios 11-300), the algorithm refined its task selection process, likely prioritizing high-reward tasks. This adjustment led to higher total rewards but also caused some low-priority tasks to be abandoned, reducing the overall completion rate. Furthermore, dynamic factors, such as task cancellations and priority changes, prompted the algorithm to respond more effectively to these fluctuations, causing significant variations in execution time for certain scenarios (e.g., Scenarios 140-150 with 150 tasks).

Overall, total rewards serve as a measure of the convergence of the proposed algorithm, reflecting the overall objective of multi-satellite dynamic scheduling. The training results for total rewards at different task scales indicate that the RH-MAPPO algorithm exhibits a commendable level of convergence.

Additionally, we designed experiments with varying rolling window sizes to evaluate the impact of the rolling horizon strategy on algorithm performance. In the experimental scheduling cycle, the satellite task scheduling system consisted of four satellites. The task simulation program generated 300 task scenarios, each containing 100 tasks, with some tasks experiencing cancellations or priority changes. The rolling window was fixed at 20 min, with rolling intervals set to 5, 10, and 15 min, and it moved forward sequentially during the scheduling cycle. The algorithm results are summarized in Table 3.

Table 3
Results of different rolling intervals of RH-MAPPO.

$W_I^{RH}(s)$	Total Time(s)	Total reward	Finished Task
300	5.96e+02	212.55	84
600	**4.90e+02**	**212.52**	**84**
900	5.07e+02	212.62	84

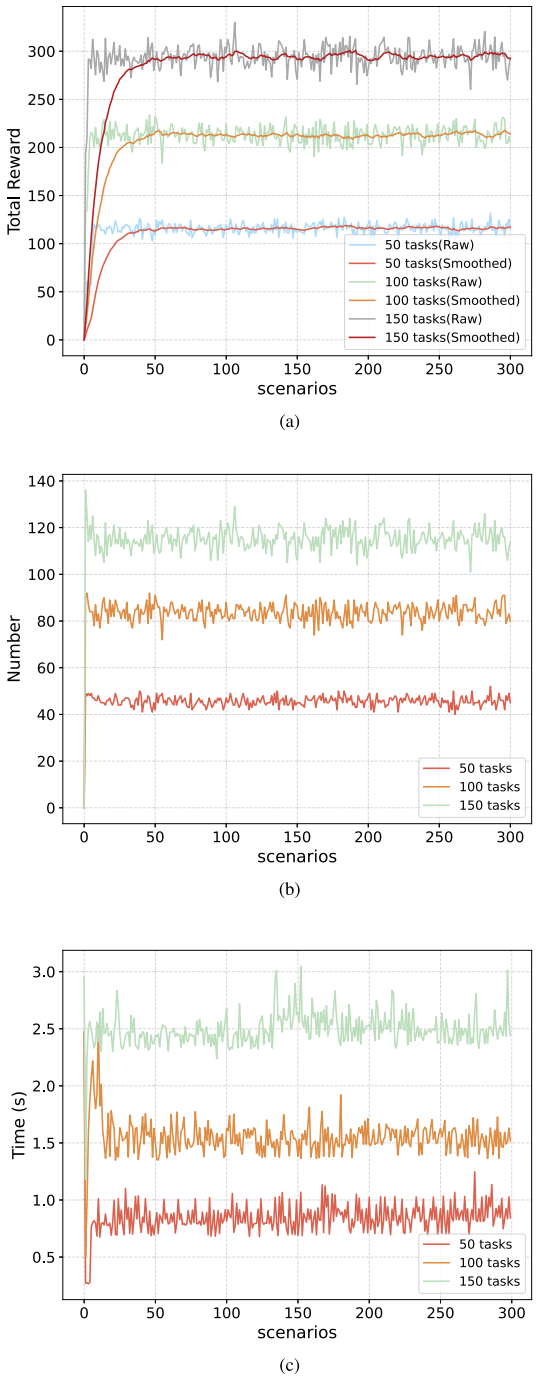

Fig. 6. Convergence curves of the RH-MAPPO algorithm for different scenario tasks. (a) Convergence curve of total reward. (b) Convergence curve for the number of successful task scheduling. (c) Average per scene of algorithm time.

The RH-MAPPO algorithm demonstrated high robustness in handling task prioritization and cancellations. Despite variations in scheduling frequency and computation time across different rolling intervals, the final rewards and the number of successfully completed tasks remained relatively stable. However, the execution time of the algorithm varied significantly. When the rolling interval was 10 min, the RH-MAPPO algorithm required the least time, approximately 490 s. In contrast, with a shorter rolling interval (5 min), the algorithm's execution time increased

Table 4
Statistics of the maximum and average per scene of profit.

Scenarios	Maximum Profit				Average Profit per scene			
	RH-MAPPO	MAPPO	MADDPG	GA	**RH-MAPPO**	MAPPO	MADDPG	GA
500_100	**302.49**	292.61	295.78	251.12	**266.57**	257.33	255.88	237.75
500_200	**536.03**	499.26	444.28	436.47	**478.22**	450.14	388.94	408.98
500_300	**650.60**	618.80	447.61	541.08	**605.97**	573.05	403.23	498.87
500_400	**735.13**	692.45	477.55	628.15	**678.92**	651.35	431.23	575.99

Table 5
Statistics of the total and average per scene of computing time.

Scenarios	Total Time(s)				Average Time per scene(s)			
	RH-MAPPO	MAPPO	MADDPG	GA	**RH-MAPPO**	MAPPO	MADDPG	GA
500_100	**1.53e+03**	1.83e+03	2.04e+03	3.41e+03	**3.06**	3.67	4.07	6.81
500_200	**2.77e+03**	3.69e+03	4.12e+03	5.97e+03	**5.54**	7.37	8.24	11.94
500_300	**4.81e+03**	5.63e+03	6.21e+03	7.69e+03	**9.61**	11.26	12.43	15.38
500_400	**6.49e+03**	7.59e+03	8.33e+03	10.01e+04	**12.98**	15.72	16.65	20.17

to nearly 600 s, likely due to greater task overlap, which required more frequent resolution of satellite resource conflicts, thereby increasing the computational burden of the scheduling system.

4.3. Algorithm performance comparison

This section compares the proposed RH-MAPPO algorithm with other deep reinforcement learning and heuristic algorithms, including MAPPO (Zhang et al., 2023), the multi-agent deep deterministic policy gradient (MADDPG) (Dalin et al., 2020), and the genetic algorithm (GA), with a focus on total profit, algorithm execution time, and task completion rate. To ensure consistency across experiments, the satellite system comprises seven satellites, and all algorithms are evaluated on 500 scenarios, each consisting of 100, 200, 300, or 400 tasks. Deep reinforcement learning algorithms are configured with identical hyperparameters, including learning rate, discount factor, experience replay buffer size, and network architecture.

4.3.1. Comparison of total profit

Total profit serves as a critical metric to evaluate both algorithm convergence and the overall quality of task scheduling. The maximum and average profits per scenario for the four algorithms are summarized in Table 4.

Across all task scales, RH-MAPPO consistently outperforms the other three algorithms in both maximum and average profits, demonstrating its superior ability to maximize returns in complex satellite scheduling scenarios. In terms of average profit per scenario, RH-MAPPO also maintains a clear lead. Although MAPPO shows moderate improvement over MADDPG as task volume increases and GA performs better than MADDPG in some settings, neither achieves the consistent high returns delivered by RH-MAPPO.

4.3.2. Comparison of the rate of finished tasks

As the task scale increases, the task completion rate becomes a key metric, reflecting the algorithm's ability to balance task prioritization and resource allocation. It is defined as the ratio of completed tasks to the total number of tasks in each scenario, calculated using the following formula:

$$fr = \frac{\text{Number of finished tasks per scenario}}{\text{Total number of scenario tasks}} \tag{29}$$

We tested 500 scenarios, and Fig. 7 summarizes the fr of the four algorithms across different scenario scales. In the 100 task scenario, the RH-MAPPO algorithm achieved full task coverage. Although the completion rate decreased as the task number increased, it remained high, particularly in the 300 task and 400 task scenarios, highlighting its adaptability to complex task environments. In contrast, MAPPO and MADDPG exhibited slower increases in task completion rates, with some scenarios showing near stagnation.

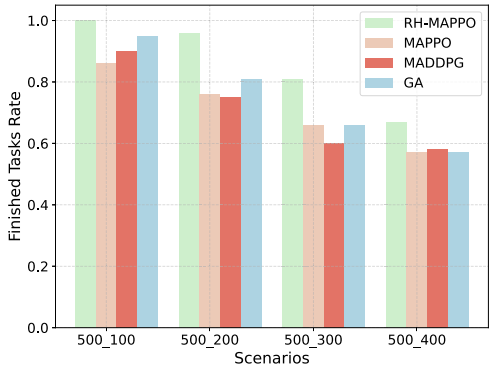

Fig. 7. Finished tasks rate of four methods in different scenario scales.

4.3.3. Comparison of computing time

This section presents the computational time of the four algorithms under varying task scales, as summarized in Table 5. Overall, RH-MAPPO consistently achieved the lowest total and average computation time across all scenarios, significantly outperforming MAPPO, MADDPG, and GA in scheduling efficiency. Notably, in the 400 task scenario, RH-MAPPO reduced total computation time by approximately 14.5 %–35.2 % compared to the other methods.

Moreover, although the computation time increased with the number of tasks for all algorithms, RH-MAPPO exhibited the slowest growth rate, indicating superior scalability and adaptability. This efficiency may arise from RH-MAPPO's ability to better adapt to dynamic task changes through the rolling horizon mechanism and its efficient resource allocation, thus minimizing scheduling time. In conclusion, RH-MAPPO demonstrates clear superiority in computational efficiency and scalability over the other algorithms.

5. Conclusion and future work

This paper proposes a novel model that formulates the distributed multi-satellite dynamic task scheduling problem as an MAMDP, addressing task dynamics in uncertain environments by treating satellites as intelligent agents capable of autonomous decision-making and environment-based feedback. This approach eliminates the need for the complex probability transition matrices typically required in traditional Markov decision processes. To solve the model, this study develops the RH-MAPPO algorithm, which seamlessly integrates the rolling horizon

strategy with MAPPO. This method dynamically balances short-term task adjustments within a rolling horizon window while maintaining long-term scheduling efficiency through cooperative decision-making among agents. By continuously adapting to real-time changes such as task priority updates and cancellations, RH-MAPPO effectively improves system responsiveness and stability under dynamic conditions. Experimental results demonstrate that RH-MAPPO outperforms other algorithms across task scales, achieving a 4.2 % improvement over MAPPO, 17.9 % over GA, and 57.3 % over MADDPG in the 400 task scenarios. It also excels in task completion and computational efficiency. Future work will address challenges like resource failures and communication interruptions, further exploring deep reinforcement learning's adaptive and fault-tolerant potential in complex task scheduling.

CRediT authorship contribution statement

Zhe Li: Methodology, Supervision, Writing – reviewing and editing; **Xin Zhu:** Algorithm design, Software implementation, Writing – original draft, Experimentation, Data curation; **Changdi Liu:** Validation, Visualization; **JunYao Song:** Investigation, Resources, Literature review; **Yubai Liu:** Data collection, Performance analysis, Visualization; **Chengjun Yin:** Conceptualization, Software; **Wei Sun:** Conceptualization, Project administration, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Campuzano-Bolarín, F., Mula, J., Díaz-Madroñero, M., & Legaz-Aparicio, Á.-G. (2020). A rolling horizon simulation approach for managing demand with lead time variability. *International Journal of Production Research*, *58*(12), 3800–3820.

Chen, S., Hu, M., & Guo, S. (2023). Fast dynamic-programming algorithm for solving global optimization problems of hybrid electric vehicles. *Energy*, *273*, 127207.

Chen, X., Wu, C., Chen, T., Zhang, H., Liu, Z., Zhang, Y., & Bennis, M. (2020). Age of information aware radio resource management in vehicular networks: A proactive deep reinforcement learning perspective. *IEEE Transactions on Wireless Communications*, *19*(4), 2268–2281.

Chu, X., Chen, Y., & Tan, Y. (2017a). An anytime branch and bound algorithm for agile earth observation satellite onboard scheduling. *Advances in Space Research*, *60*(9), 2077–2090.

Chun, J., Yang, W., Liu, X., Wu, G., He, L., & Xing, L. (2023). Deep reinforcement learning for the agile earth observation satellite scheduling problem. *Mathematics*, *11*(19), 4059.

Cordeau, J.-F., & Laporte, G. (2005). Maximizing the value of an earth observation satellite orbit. *Journal of the Operational Research Society*, *56*(8), 962–968.

Cui, R., Han, W., Su, X., Zhang, Y., & Guo, F. (2020). A multi-objective hyper heuristic framework for integrated optimization of carrier-based aircraft flight deck operations scheduling and resource configuration. *Aerospace Science and Technology*, *107*, 106346.

Dai, C.-Q., Li, C., Fu, S., Zhao, J., & Chen, Q. (2020). Dynamic scheduling for emergency tasks in space data relay network. *IEEE Transactions on Vehicular Technology*, *70*(1), 795–807.

Dalin, L., Haijiao, W., Zhen, Y., Yanfeng, G., & Shi, S. (2020). An online distributed satellite cooperative observation scheduling algorithm based on multiagent deep reinforcement learning. *IEEE Geoscience and Remote Sensing Letters*, *18*(11), 1901–1905.

Dell'Amico, M., Montemanni, R., & Novellani, S. (2021). Algorithms based on branch and bound for the flying sidekick traveling salesman problem. *Omega*, *104*, 102493.

Dey, S. S., Dubey, Y., & Molinaro, M. (2023). Lower bounds on the size of general branch-and-bound trees. *Mathematical Programming*, *198*(1), 539–559.

Eddy, D., & Kochenderfer, M. (2020). Markov decision processes for multi-objective satellite task planning. In *2020 IEEE Aerospace conference* (pp. 1–12). IEEE.

Efrem, C. N., & Panagopoulos, A. D. (2019). Dynamic energy-efficient power allocation in multibeam satellite systems. *IEEE Wireless Communications Letters*, *9*(2), 228–231.

Fan, H., Yang, Z., Zhang, X., Wu, S., Long, J., & Liu, L. (2022). A novel multi-satellite and multi-task scheduling method based on task network graph aggregation. *Expert Systems with Applications*, *205*, 117565.

Ferrari, B., Cordeau, J.-F., Delorme, M., Iori, M., & Orosei, R. (2024). Satellite scheduling problems: A survey of applications in earth and outer space observation. *Computers & Operations Research*, *173*, 106875.

Ferreira, P. V. R., Paffenroth, R., Wyglinski, A. M., Hackett, T. M., Bilen, S. G., Reinhart, R. C., & Mortensen, D. J. (2019). Reinforcement learning for satellite communications: From LEO to deep space operations. *IEEE Communications Magazine*, *57*(5), 70–75.

Filar, J., & Vrieze, K. (2012). Competitive Markov decision processes. Springer Science & Business Media.

Gkiotsalitis, K. (2021). Bus rescheduling in rolling horizons for regularity-based services. *Journal of Intelligent Transportation Systems*, *25*(4), 356–375.

Hassan, S. S., Park, Y. M., Tun, Y. K., Saad, W., Han, Z., & Hong, C. S. (2023). Satellite-based ITS data offloading & computation in 6G networks: A cooperative multi-agent proximal policy optimization DRL with attention approach. *IEEE Transactions on Mobile Computing*. *23*(5), 4956–4974.

He, L., Liu, X.-L., Chen, Y.-W., Xing, L.-N., & Liu, K. (2019a). Hierarchical scheduling for real-time agile satellite task scheduling in a dynamic environment. *Advances in Space Research*, *63*(2), 897–912.

He, Y., Chen, Y., Lu, J., Chen, C., & Wu, G. (2019b). Scheduling multiple agile earth observation satellites with an edge computing framework and a constructive heuristic algorithm. *Journal of Systems Architecture*, *95*, 55–66.

He, Y., Xing, L., Chen, Y., Pedrycz, W., Wang, L., & Wu, G. (2020). A generic markov decision process model and reinforcement learning method for scheduling agile earth observation satellites. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, *52*(3), 1463–1474.

Heydari, A. (2020). Optimal impulsive control using adaptive dynamic programming and its application in spacecraft rendezvous. *IEEE Transactions on Neural Networks and Learning Systems*, *32*(10), 4544–4552.

Hu, Q., & Yue, W. (2007). Markov decision processes with their applications (vol. 14). Springer Science & Business Media.

Kang, H., Chang, X., Mišić, J., Mišić, V. B., Fan, J., & Liu, Y. (2023). Cooperative UAV resource allocation and task offloading in hierarchical aerial computing systems: A MAPPO-based approach. *IEEE Internet of Things Journal*, *10*(12), 10497–10509.

Kumar, A., Gaur, N., Chakravarthy, S., & Nanthaamornphong, A. (2025). Enhancing satellite networks with deep reinforcement learning: A focus on iot connectivity and dynamic resource management. *Results in Optics*, *18*, 100765.

Lemaître, M., Verfaillie, G., Jouhaud, F., Lachiver, J.-M., & Bataille, N. (2002). Selecting and scheduling observations of agile satellites. *Aerospace Science and Technology*, *6*(5), 367–381.

Li, H., Li, Y., Liu, Y., Zhang, K., Li, X., Li, Y., & Zhao, S. (2024a). A multi-objective dynamic mission-scheduling algorithm considering perturbations for earth observation satellites. *Aerospace*, *11*(8), 643.

Li, P., Wang, H., Zhang, Y., & Pan, R. (2024b). Mission planning for distributed multiple agile earth observing satellites by attention-based deep reinforcement learning method. *Advances in Space Research*. *74*(5), 2388–2404.

Li, X., Sun, C., Fan, H., & Yang, J. (2024c). Remote-sensing satellite mission scheduling optimisation method under dynamic mission priorities. *Mathematics*, *12*(11), 1704.

Liang, J., Liu, J.-P., Sun, Q., Zhu, Y.-H., Zhang, Y.-C., Song, J.-G., & He, B.-Y. (2023). A fast approach to satellite range rescheduling using deep reinforcement learning. *IEEE Transactions on Aerospace and Electronic Systems*. *59*(6), 9390–9403.

Mi, X., Song, Y., Yang, C., Han, Z., & Yuen, C. (2024). MAGIC: Matching game-based resource allocation with incomplete information in space communication network. *IEEE Transactions on Communications*. *72*(6), 3481–3494.

Qiu, W., Xu, C., Ren, Z., & Teo, K. L. (2021). Scheduling and planning framework for time delay integration imaging by agile satellite. *IEEE Transactions on Aerospace and Electronic Systems*, *58*(1), 189–205.

Ren, L., Ning, X., & Wang, Z. (2022). A competitive markov decision process model and a recursive reinforcement-learning algorithm for fairness scheduling of agile satellites. *Computers & Industrial Engineering*, *169*, 108242.

Shi, W., Zhang, D., Han, X., Wang, X., Pu, T., & Chen, W. (2023). Coordinated operation of active distribution network, networked microgrids, and electric vehicle: A multi-agent PPO optimization method. *CSEE Journal of Power and Energy Systems*. 1–12.

Song, Y., Ou, J., Pedrycz, W., Suganthan, P. N., Wang, X., Xing, L., & Zhang, Y. (2024). Generalized model and deep reinforcement learning-based evolutionary method for multitype satellite observation scheduling. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, *54*(4), 2576–2589.

Sutton, R. S. (2018). Reinforcement learning: An introduction. *A Bradford Book*, .

Wang, X., Wu, G., Xing, L., & Pedrycz, W. (2020). Agile earth observation satellite scheduling over 20 years: Formulations, methods, and future directions. *IEEE Systems Journal*, *15*(3), 3881–3892.

Wei, L., Xing, L., Wan, Q., Song, Y., & Chen, Y. (2021). A multi-objective memetic approach for time-dependent agile earth observation satellite scheduling problem. *Computers & Industrial Engineering*, *159*, 107530.

Wolfe, W. J., & Sorensen, S. E. (2000). Three scheduling algorithms applied to the earth observing systems domain. *Management Science*, *46*(1), 148–166.

Wu, O., Dalle Ave, G., Harjunkoski, I., & Imsland, L. (2021). A rolling horizon approach for scheduling of multiproduct batch production and maintenance using generalized disjunctive programming models. *Computers & Chemical Engineering*, *148*, 107268.

Wu, Y., Xiao, L., Zhou, J., Feng, M., Xiao, P., & Jiang, T. (2024). Large-scale MIMO enabled satellite communications: concepts, technologies, and challenges. *IEEE Communications Magazine*, *62*(8), 140–146.

Yuan, S., Peng, M., Sun, Y., & Liu, X. (2023). Software defined intelligent satellite-terrestrial integrated networks: Insights and challenges. *Digital Communications and Networks*, *9*(6), 1331–1339.

Zarpellon, G., Jo, J., Lodi, A., & Bengio, Y. (2021). Parameterizing branch-and-bound search trees to learn branching policies. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 3931–3939). (*vol. 35*).

Zhang, G., Li, X., Hu, G., Li, Y., Wang, X., & Zhang, Z. (2023). MARL-based multi-satellite intelligent task planning method. *IEEE Access*, *11*, 135517–135528.

Zhang, J., Xing, L., Peng, G., Yao, F., & Chen, C. (2019). A large-scale multiobjective satellite data transmission scheduling algorithm based on SVM+ NSGA-II. *Swarm and Evolutionary Computation*, *50*, 100560.

Zheng, Z., Guo, J., & Gill, E. (2019). Distributed onboard mission planning for multi-satellite systems. *Aerospace Science and Technology*, *89*, 111–122.